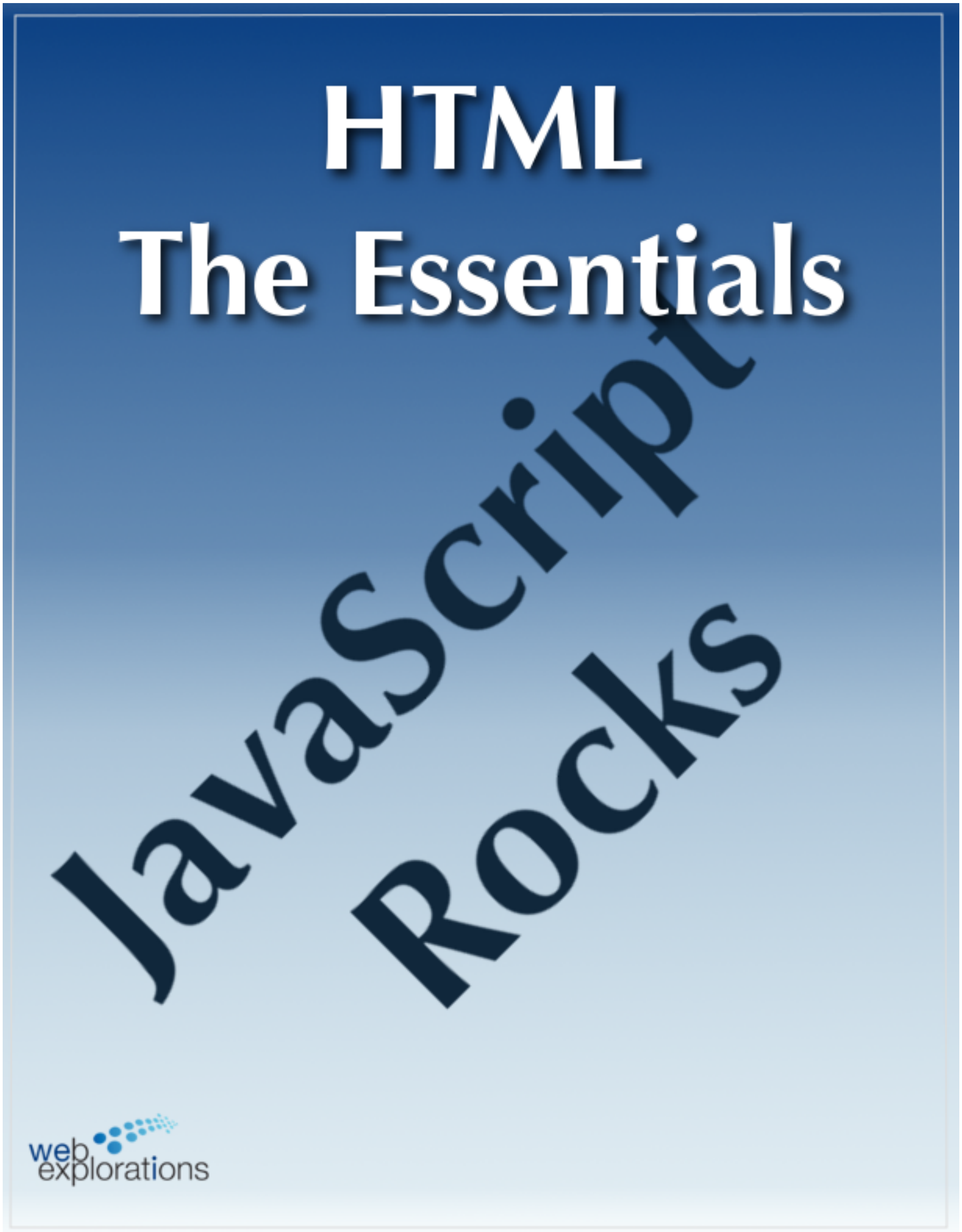

HTML
The Essentials
JavaScript Rocks
web explorations

Get the Tools

Introduction

This tutorial will have you creating web pages in just a few minutes.

First, you'll learn the secret character that every browser looks for.
Next, you'll learn how to create an HTML file using special tags and how to view the page.

You'll end up with a template that you can use to create new web pages in the future.

You only need two tools to create web pages

A Web Browser

Any modern browser will work. Most developers work using FireFox and then check their pages later using Google Chrome, Internet Explorer, Safari (Macintosh only), and Opera.

A Text Editor

Notepad++ is a free, open-source text editor for Windows.

TextWrangler, from BareBones Software, is a free text editor that you can run on a Mac.

The Secret Character

A browser is simply a program written to look for a secret character inside a text file and then changing the markup or display of the text that follows.

The secret character is the less than "<" bracket.
In the tag above the browser knows to start treating the text that follows as html code or web page.

As the browser reads through a web page it looks at each character to see if it is a <. If so it knows that it found a markup tag. It reads in the whole tag until it finds the greater than ">" character and then the browser looks up the command in its dictionary so it knows what to do with the text that follows.

The browser keeps handling that until a / or stop character is found. </html> tells the browser to stop doing <html> stuff.

Here is another example from an HTML file:

```
This is a simple sentence <strong>with this text being
bold</strong> and all the other text is normal.
```

In a browser this code will look like this:

This is a simple sentence **with this text being bold** and all the other text is normal.

Notice that the element has disappeared.

Once the browser found the secret character "<" and determined to make the text bold, it throws away the tag.
When the browser came on the "stop strong" the browser threw that tag away as well and returned to showing regular text.

Some things to remember:

- Every start tag should have a stop tag.   and
- If a tag is spelled wrong the browser will just ignore it and throw the start and stop tags away.
- Two tags and all the stuff in between is called an element.

If you are curious, here is a complete list of all the approved HTML 5 elements from W3Schools.com

Build a Page

Open up your favorite text editor.

Using File/New from the menu, create a new file.
Be smart and save it right away (CTRL s) making sure the file type is .html.

You might want to call this file: firstPage.html

Type in the following code:

```
<!doctype html>
<html>
<head>
 <title>My First WebPage</title>
</head>
<body>
  <p>
  This is a simple sentence <strong>with this text being bold</strong> and all
    the other text is normal.
  </p>
</body>
</html>
```

Displays in the tab of the browser window.

Everything in the <body> displays on the page.

Things to pay attention to:

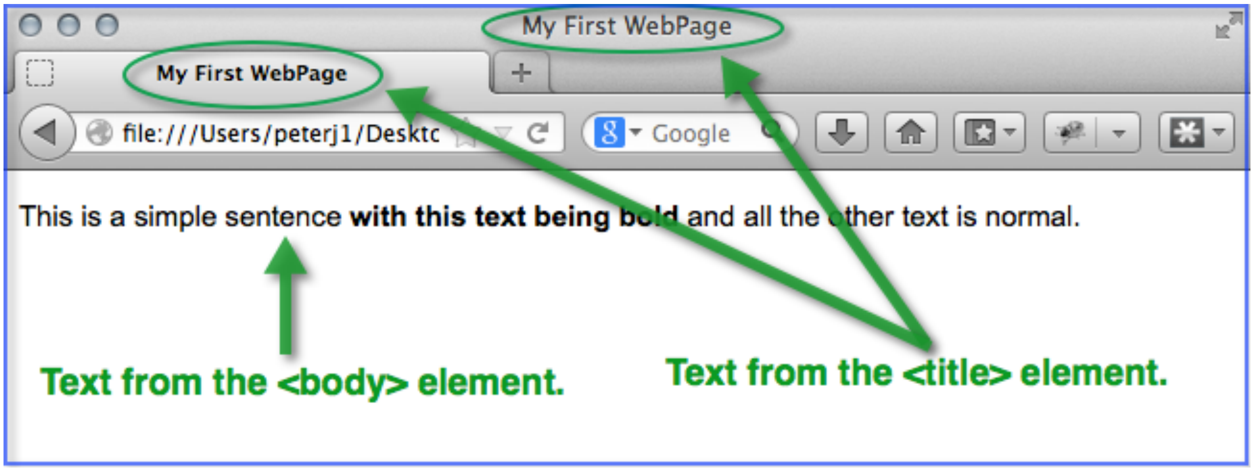
- If the tags are not color coded then you probably saved the file as a .txt file. Text editors use the file extension to determine how to color code the keywords. (Your color coding may be different than this screenshot. Each text editor uses its own color scheme.)
- All the html tags are always typed using lower case.
- <title> means title bar. NOT the title of the page. The text in the <title> element only shows up in the browser tabs at the title bar of the browser window.
- The <p> element creates a paragraph block with a block line in between each one.
- Every page you create should have it's own unique <title>. That is how we tell pages apart when we we have several pages open in a browser.

Open up your favorite browser.

Keep your text editor open because you will be jumping back and forth between the two programs.

Use File/Open from the browser menu bar (CTRL o - oh for Open, not zero) and select your firstPage.html file.

It should look something like this:



Headings and Line Breaks

People scan web pages.

So, a smart web developer learns to include headings on the page highlighting the big ideas.

These range from <h1> for the main idea all the way down to a sub-sub-sub-sub-sub-head or <h6>.

Working in your text editor, add this code to your web page typing in the lines exactly as shown:

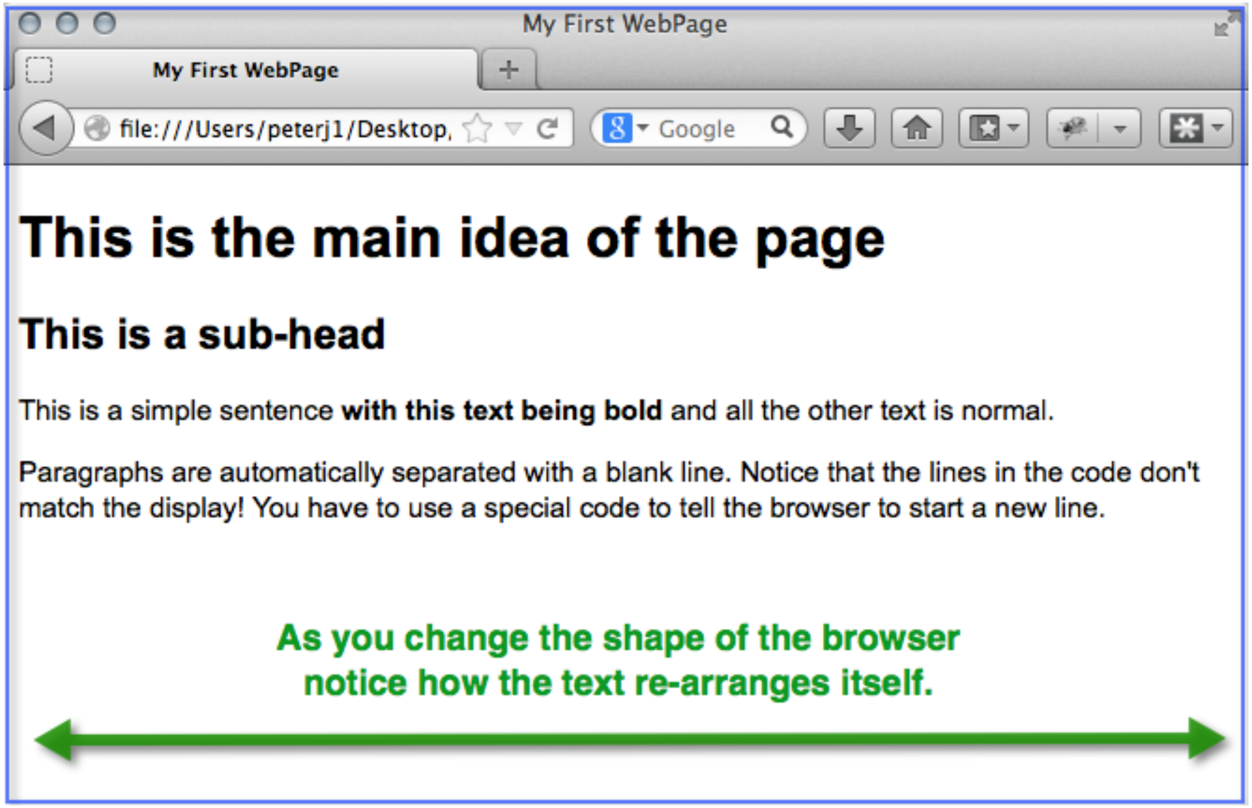
```
<!doctype html>
<html>
<head>
 <title>My First WebPage</title>
</head>
<body>
   <h1>This is the main idea of the page</h1>
   <h2>This is a sub-head</h2>
   <p>
   This is a simple sentence <strong>with this text being bold</strong> and all the other text is normal.
   </p>

   <p>
   Paragraphs are automatically separated with a blank line.
   Notice that the lines in the code
   don't match the display!
   You have to use a special code to tell the browser to start a new line.
   </p>

</body>
</html>
```

Don't forget to save the file (CTRL s)!

Now, shift over to your browser and refresh the page using CTRL r.
It should look something like this:



Notice that the <h1> text is larger then the <h2> and that they are both display as bold text automatically. (In the future you will learn how to change this appearance using CSS).

As you change the shape of the browser notice how the text automatically flows and wraps around. This makes you page flexible because you can't tell what size browser someone is using. It could be small and narrow like a mobile device or wide and spacious like a high-resolution desktop computer.

This allows you to format the HTML code for maximum readability. And, if your code is nicely formatted with indenting it will be much easier for you, or someone else to maintain it in the future.

Force the browser to start a new line

But, what if you want to start a new line? That's where the line break
 element is useful. It tells the browser that no matter what, add in a line break.

Add in some line breaks
 to your page. (You might even want to experiment and add one right in the middle of a line, like right after the words "special code" on line #17.)

```
1  <!doctype html>
2  <html>
3  <head>
4   <title>My First WebPage</title>
5  </head>
6  <body>
7    <h1>This is the main idea of the page</h1>
8    <h2>This is a sub-head</h2>
9    <p>
10   This is a simple sentence <strong>with this text being bold</strong> and all
       the other text is normal.
11   </p>
12
13   <p>
14   Paragraphs are automatically separated with a blank line.<br />
15   Notice that the lines in the code<br />
16   don't match the display!<br />
17   You have to use a special code to tell the browser to start a new line.
18   </p>
19
20 </body>
21 </html>
```

While you are at it, experiment a bit and add a horizontal rule element to your page as well.

```
<hr />
```

You could put it after the first </p> "stop paragraph" tag to separate the two paragraphs visually on the page.

When you view the page it should look something like this even if you make the browser window wider:

Why does the
 element look so weird?

Do you remember at the beginning of this tutorial, that every start tag needs a matching stop tag?
Well, the
 started out as a single tag. In order to follow the rules we add in a space stop slash " /" to let the browser know that that is the end of the line break element.

Although the majority of HTML elements have a separate start and stop tag (like <html> and </html> and <p> and </p>) there are a few single tag elements. Each of them should end with a space end-slash greater than bracket: " />.

-
 Adds a line break
- <hr /> Adds a horizontal rule
- <img /> -Displays graphic files on the web page
- <meta /> adds information in the <head> element about the page.
- <input /> used to add form items, like a text box or button, on a page.

Setting up a Programming Cycle

Hot Keys Help You Work Faster

As you build your web pages you will soon fall into a programming routine:

1. Add some text in the text editor.
2. Save the file.
3. Switch to the browser.
4. Refresh the page to see the results.
5. Switch back to the text editor
6. Continue the loop.

Being able to see your changes right away is one of the things that makes programming so much fun.

Keep in mind that when you use a mouse your brain is using different parts of its brain then when you are using the keyboard. As you shift from the mouse to the keyboard and back to the mouse your hands have to reposition themselves as well as your brain. That all takes time.

Smart programmers start using hot keys right away making the edit/save/switch/refresh/switch cycle much faster.

Here are the steps with the hot keys added. (On the Mac use the Command key instead of CTRL)

1. Add some text in the text editor.
2. Save the file. CTRL s
3. Switch to the browser. ALT tab
4. Refresh the page to see the results. CTRL r
5. Switch back to the text editor ALT tab
6. Continue the loop.

If you change something that you want to undo, just use the universal CTRL z.

You might have to practice with the ALT tab a bit.

If you hold the ALT (Command on the Mac) key down with your thumb and tap the TAB key once, and release the CTRL key, the last program will become active.

If you hold the ALT key down AND hold the TAB key down all the active program icons will display across the screen.

Each time you tap the TAB key it will jump to the next icon. When you have the application icon highlighted, release the ALT key and that app will display.

Remember that if you are only working with two applications such as your text editor and the browser all you have to do is hit the ALT TAB combination once and the previous program will automatically become active. This lets you quickly toggle back and forth between the two programs: editor/browser/editor/browser/sweet programming!

DOCTYPE and meta tags

There are several different versions of HTML including XHTML, HTML 4.0, and the newest, HTML 5.

Using the <!doctype> element at the very beginning of the document tells the browser exactly which version of HTML it should be using.

For HTML 5 websites (like you are creating right now) simply put this on the first line of your page:

```
<!doctype html>
```

And, because the Web is global the browser needs to know which character set should be used. This is called encoding. This is done using a <meta /> element like this:

```
 <meta charset="utf-8" />
```

Using UTF-8 allows browsers to convert the existing information on the page into other languages such as Chinese or Arabic.

Notice that the <meta /> tags are single-tag elements. You need to include the space slash greater than " />" to close the tag off.

Also, the <meta /> tags always go inside the <head> element. They can be before or after the <title> element but all <meta /> elements must be inside the <head> section.

Add the two lines to your test page:

```
<!doctype html>
<html>
<head>
 <title>My First WebPage</title>
 <meta charset="utf-8" />
</head>
<body>
  <h1>This is the main idea of the page</h1>
  <h2>This is a sub-head</h2>
  <p>
  This is a simple sentence <strong>with this text being bold</strong> and all
    the other text is normal.
  </p>

  <p>
  Paragraphs are automatically separated with a blank line.<br />
  Notice that the lines in the code<br />
  don't match the display!<br />
  You have to use a special code to tell the browser to start a new line.
  </p>

</body>
</html>
```

Add the doctype on the very first line.

Add a meta tag with the charset=" " attribute inside the <head> element.

If you view the page in the browser it will appear unchanged. However, now any browser will know what version to use to display the contents and HTML codes on the page.

Comments to your future self

Comments are notes to your future self.

As a web developer you will want to leave lots of notes to yourself (and anyone that has to work with your code in the future.) But, you don't want this information to actually display on the web site.

You can do this by using the comment tags:

```
<!--  and -->
```

As soon as the browser sees <!-- it knows to ignore all the text until it sees the stop comment: -->

The only thing you can't put inside a comment is a double dash "--" because it confuses the browser into thinking that is the end of the commented text.

Professional programmers always put a comment block at the top of their code to identify the file, who wrote it, and most importantly, a list of all revisions that have been made to the page.

It is important that the browser know what the charset is before having to handle too much information so it is smart to always put the comment block AFTER the <meta charset="utf-8" /> element.

Notice the spaces separating the comment tags from the text.

```
<!doctype html>
<html>
<head>
 <title>My First WebPage</title>
 <meta charset="utf-8" />
 <!-- firstPage.html - My first HTML web page
      Written by:   Jose Mendez   Jose.Mendez60@gmail.com
      Written:  10-16-14
      Revised:  JM 12-04-14
 -->
</head>
<body>
  <!-- The text that will display in the browser -->
  <h1>This is the main idea of the page</h1>
  <h2>This is a sub-head</h2>
  <p>
  This is a simple sentence <strong>with this text being bold</strong> and all the other text is normal.
  </p>

  <p>
  Paragraphs are automatically separated with a blank line.<br />
  Notice that the lines in the code<br />
  don't match the display!<br />
  You have to use a special code to tell the browser to start a new line.
  </p>

</body>
</html>
```

Comments can be multiple lines.

or single lines

Most text editors display comments in green. If your editor doesn't you can always go into the Preferences or Setup and set the color of all comments to green.

Comment tags work with both multiple-line comments as well as single line comments.

Be nice to all browsers and separate the comment markers and the text with spaces.

Validating Your Code

Check your work!

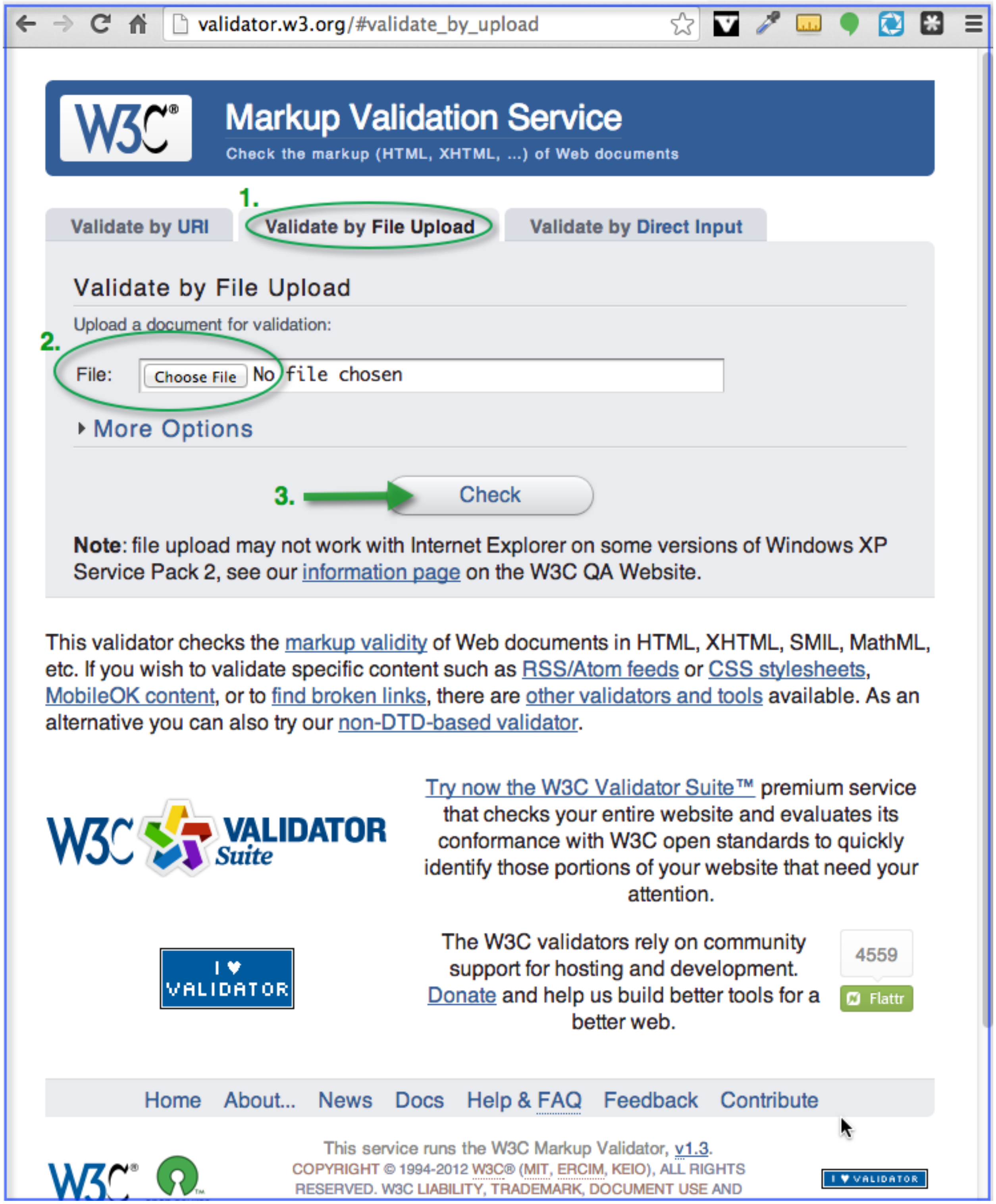
You've learned several HTML elements and specialized tags. How do you know your page meets the HTML 5 standard?

You can validate the code on your page by using the HTML Validator.

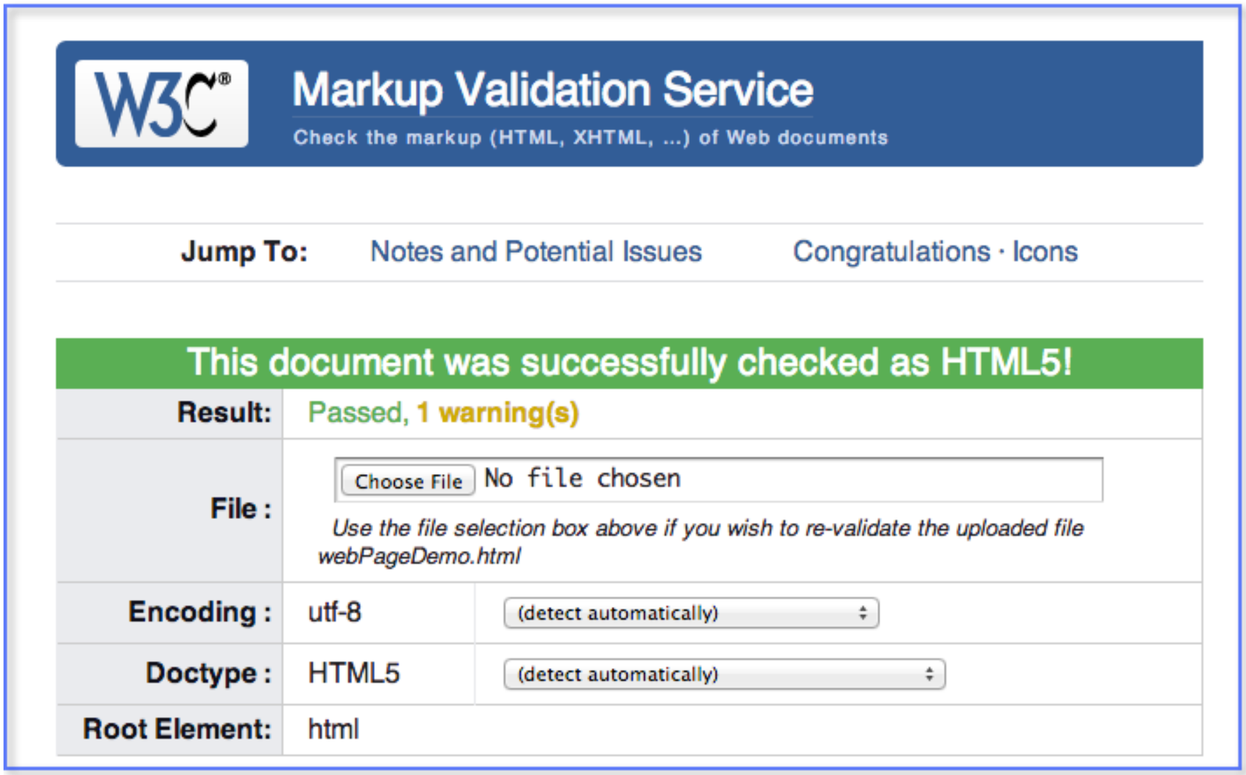
The World Wide Web Consortium (W3C) has established all the rules for each version of HTML and they have an online validator you can use.

http://Validator.w3.org/#validate_by_upload

Make sure you select the "Validate by File Upload" tab. Then choose the .html file you want to validate and click the "Check" button.



You want to see the green bar "This document was successfully checked as HTML5!"



Other validation tools you can use.

The WDG Web Design Group also allows you to upload your .html file for validation. http://www.htmlhelp.com/tools/validator/upload.html.en

There are also websites that allow you to copy your code and paste it into a window for validation as well as to reformat it so the code is easier to work with.
HTML Tidy Online at http://infohound.net/tidy/ is one of these validation sites.

You can find others by doing a Google search for: HTML online validation

Creating and Using a Template

You now have the basic elements for any web page.

Instead of retyping all of this code every time you want to start a new website, save this file as a template and use it as a starting point for all your future web pages.

A good trick is to replace the key items with ??? (easy to find using CTRL f) and name the file template.html.

To use it simply open up the file in your text editor and immediately do a File/Save As saving it with a new file name in your working directory.

Here is what your template.html might look like:

template.html

```
<!doctype html>
<html>
<head>
 <title>???</title>
 <meta charset="utf-8" />
 <!-- ???.html - My first HTML web page
      Written by:   Jose Mendez  Jose.Mendez60@gmail.com
      Written:  ???
      Revised:
 -->
</head>
<body>
  <!-- The text that will display in the browser -->
  <h1>???</h1>
  <p>
  This is a simple sentence and all the other text is normal.
  </p>
</body>
</html>
```

How the Web Works

Have you noticed you didn't need the Web?

A browser is simply a program that reads text from a file using special markup tags to determine how the text should be displayed on the page.

So, we didn't need the Web at all. But... without the Web, only you, or someone at your computer can view the page. That's not really a lot of fun.

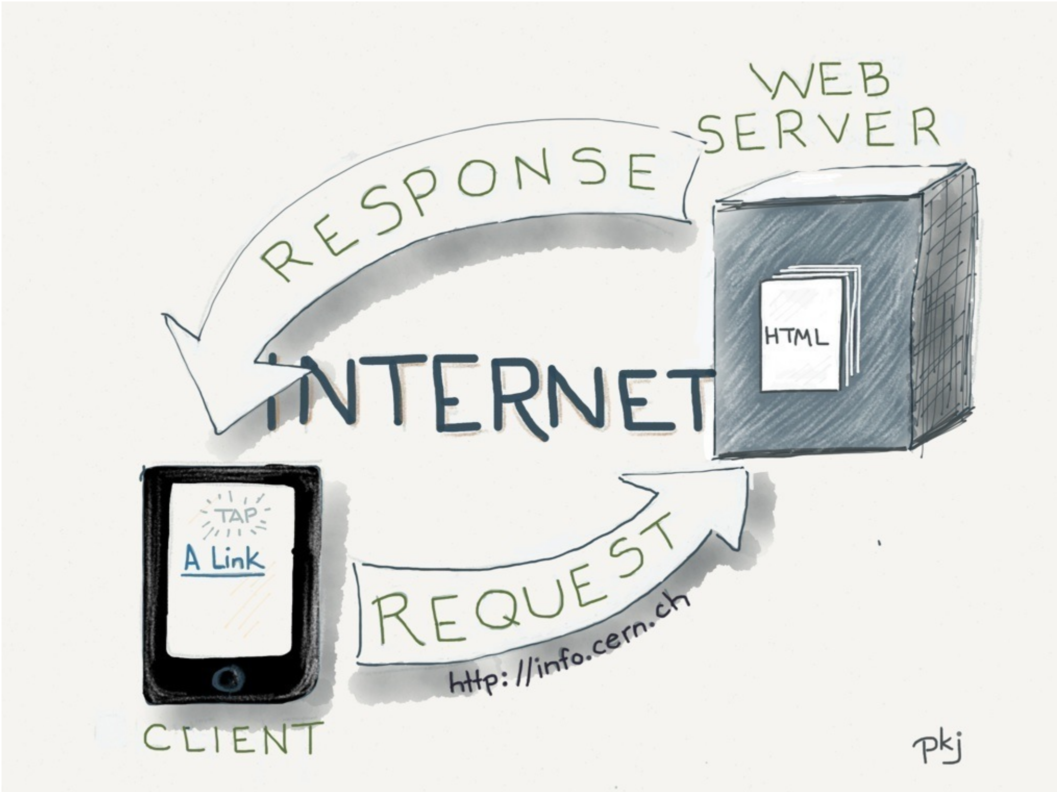
By putting your file out on a network (i.e. the Internet) other people can request a copy of your web page and then view it in their very own browser. Suddenly, instead of just you being able to view the file, millions of people from all over the world can view it in seconds.

Need to make a change? No problem. Just update the file on the web server and everyone that sees the file after that sees the changes immediately.

Here is how the Web works

When a user clicks a link or types in a web address (URL) the browser sends a REQUEST to the web server.

A web server is simply a computer with lots of files that sits and waits for browsers to request things.



The web server locates the file on it's system, makes a copy of it and then sends back a RESPONSE back to the server. The response is a series of data packets with overlapping information in case one or more of them gets destroyed or lost out on the Internet.

When the browser receives the file it reads in all the text and uses the tags to decide how to display it for the user. This exchange of information, which can happen on the other side of the world, happens in seconds.

This type of REQUEST/RESPONSE is called a client/server relationship. The client is the browser and the server is the web server.

Who designed all this?
Tim Berners-Lee

Around 1990, Tim Berners-Lee, a British scientist, was working at CERN labs in Geneva, Switzerland. He came up with the idea of sharing files on a server using a special markup language originally created by IBM. He talked about his idea but the other people he worked with didn't see the advantage. After all, they had email and could send files back and forth across the network, what did they need these shared pages for?

The laboratories at CERN had a lot of scientists coming and going and the mailing list was always out of date. Tim Berners-Lee put the mailing list on his web server and allowed several of the other scientists to access these pages using the basic web browser program that Tim had written as a side project. Whenever someone arrived or left CERN all Tim had to do was change the one file on his server and everyone with access to the web server would immediately have the up-to-date version of the mailing list.

Suddenly the people around Tim understood what a great communication tool this would be for everyone.

Here is a link to the original web site Tim Berners-Lee created at CERN.

CERN has preserved this site and the first domain name http://info.cern.ch and has written more history about the development of the Web.

Tim Berners-Lee wanted to improve communications around the world between scientists and universities. Later, he founded the World Wide Web Consortium (W3C). This non-profit organization brings in all the major players (Microsoft, IBM, Google, et al) so they can decide on a standards for web technologies. This is where the HTML 5 standard that you are using comes from. Here is a list of the people on the HTML 5 working group.

Best Practices

Here is a list of best practices that will help you create quality web pages

Use consistent naming conventions

Web servers are very picky on spelling as well as UPPER/lower case. myPhoto.jpg, my photo.jpg, and MyPhoto.JPG are all very different filenames. If you name a file myPhoto.jpg on the server and then refer to it as MyPhoto.jpg in your code, then the web browser will tell you that it can't find the file.

Here is a set of rules that you can use to ensure that all your filenames will be recognized.

1. Always start with a lower case letter. For example: myPhoto.jpg
2. Don't use spaces or underscores - Web servers change spaces into %20 and underscores are hidden by the underline used for hyperlinks.
3. useCamelCaseForReadability - Make the first letter of each word upper case. All the others keep lower case. Acronyms such as IBM should be kept upper case. For example: timBernersLee.jpg
4. Extensions should always be lower case. For example: .html .jpg .gif .png. Watch out for photo editing programs. Some of them just love to save the file extension as UPPER case letters. (Don't put extensions on folders, just file names.)
5. Be consistent. Follow these naming rules for all of your file and folder names.

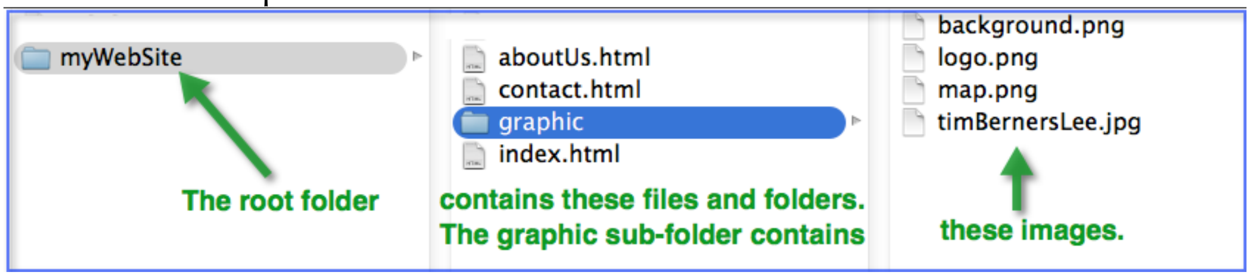
Use agile development: Write a little code then view it, then write some more, then view it.

Check your code often. It is much easier to find out errors from a few lines of code then it is from a whole page. To help along the way, validate your code often. Many programmers put a shortcut to the validation website on the link bar of their browser. Then, as they develop a page they can quickly go out and validate the code.

Stay organized.

Set up a folder structure so each project or web site has its own folder. Each folder should hold your web pages as well as a graphic sub-folder to hold all the graphic images used on that particular site.

Here is an example:



Validate your code often.

Let the HTML Validator catch any typos or HTML errors right away.

If you web pages use valid code you will get better search engine results and you will know that your pages will display no matter what browser people are using.

Summary

What have you learned?

You now have a working HTML 5 template file.

Here are some of the things that you have learned:

- The secret character that every browser program looks for.
- The two tools needed to create web pages.
- The five essential HTML elements in the correct order:

```
<!doctype html>
<html>
<head>
      <title>Text for the title and tabs goes
here</title>
</head>
<body>
            Everything displaying in the browser
window goes here.
</body>
</html>
```

- The character set UTF-8 and how to include it in a <meta /> element.
- Other common elements include:

```
<h1> <h2> <h3> <h4> <h5> <h6>
<strong>
<p>
<br />
<hr />
```

- The difference between double tag elements and single tag elements
- How to add hidden programming comments on a page.

- How to validate your HTML code.
- How the Web works.
- Who created the Web.

Credits

HTML, The Essentials, A JavaScript Rocks! tutorial written by Peter K. Johnson
http://WebExplorations.com

Photo Credits

NotePad++ logo courtesy of Don Ho http://notepad-plus-plus.org/ - open-source software.
TextWrangler logo courtesy of Bare Bones Software - http://www.barebones.com/products/textwrangler/
I Love Alt Tab t-shirt image courtesy of cafe press - http://www.cafepress.com/thealttab
Portrait of Tim Berners-Lee from Flickr.com by knightFoundation http://www.flickr.com/photos/knightfoundation/2467553359/ Creative Commons License
Portrait of Tim Berners-Lee with first web site by CERN from Flickr.com by NO4 http://www.flickr.com/photos/12693492@N04/1339026964/ Creative Commons License

Written: December 3, 2013

web explorations